

CSE 3216 - Software Design Patterns Lab

Project Report



Project Name:
**MediSched+: Online Healthcare Appointment
Management System**

Team Name : MediDesigners

Submitted To:
Md Mahmudur Rahman, Assistant Professor, CSEDU
Md. Fahim Arefin, Lecturer, CSEDU

Submitted By:
Abdullah Al Arman Emon (50)
Shah Jamal Islam (81)
Suhail Tanvir Nahin (82)
Md. Rakib Hossain (88)

28th Batch

Department of Computer Science and Engineering
University of Dhaka

Contents

1	Introduction	3
1.1	Problem Definition & Context	3
1.2	Motivation	3
1.3	Core Features	3
1.4	Tools, Technologies & Frameworks Used	7
1.5	Individual Responsibilities	7
2	System Overview	7
2.1	System Architecture	8
2.2	Use Case Diagrams	10
3	Design Patterns Used	18
3.1	Design Philosophy & Rationale	18
3.2	Factory Method Pattern	19
3.3	Strategy Pattern	21
3.4	Observer Pattern	22
3.5	State Pattern	24
3.6	Decorator Pattern	26
3.7	Template Method Pattern	28
4	System Implementation	29
4.1	UI Screens	29
4.2	Code Modules & Responsibilities	30
4.3	File / Package Structure	30
4.4	Database Schema	31
4.5	Code Repository Link	31
4.6	Video Demo Link	32
5	Evaluation	32
5.1	Evidence of Improvement	32
5.2	Comparison With Non-Pattern Alternative	32
5.3	Maintainability Assessment	33
1	UML Diagrams	34
1.1	Factory Method Pattern - Class Diagram	34
1.2	Strategy Pattern - Class Diagram	34
1.3	Observer Pattern - Class Diagram	35
1.4	State Pattern - Class Diagram	35
1.5	Decorator Pattern - Class Diagram	36
1.6	Template Method Pattern - Class Diagram	36
2	Glossary	37
3	References	38

1 Introduction

1.1 Problem Definition & Context

The project, **MediSched**, addresses the challenges in traditional healthcare appointment systems, which often suffer from long waiting times, geographical barriers, and inefficient communication between patients and doctors. The context is a modern, web-based platform designed to streamline the process of scheduling, managing, and conducting virtual medical consultations. It aims to provide a robust, multi-user environment for patients to connect with specialized doctors for both general and special consultations, complete with integrated payment and communication features.

1.2 Motivation

The primary motivation for developing MediSched was to apply software design patterns to create a highly maintainable, scalable, and extensible application. The goal was to move beyond simple CRUD (Create, Read, Update, Delete) operations and implement established architectural and object-oriented patterns to handle complex features like user role management, external API integration (for video calls), and object creation logic in a clean, decoupled manner. This ensures the system can easily adapt to new requirements, such as adding new user roles or integrating different communication platforms, without major refactoring.

1.3 Core Features

The MediSched platform incorporates the following core features:

1.3.1 Multi-Role Authentication

The system implements a robust authentication mechanism that supports two distinct user roles: Patient and Doctor. Each role has specific permissions and access levels, ensuring that users can only perform actions relevant to their role. The authentication system is built using Django's built-in authentication framework with a custom `CustomUser` model that extends `AbstractUser`.

Key Aspects:

- Role-based user registration and login
- Secure password management with Django's built-in validators
- Session-based authentication for maintaining user state
- Custom decorators for role-based access control

1.3.2 Doctor Profile Management

Doctors can create and manage comprehensive profiles that include their qualifications, experience, specialization, and contact information. The profile management system allows doctors to showcase their expertise and credentials to potential patients.

Key Aspects:

- One-to-one relationship between `CustomUser` and `Doctor` model
- Fields for qualifications, BMDC number, total experience, and working hours
- Image upload for profile picture
- Specialization management through `DoctorSpecializationDepartment` and `DoctorSpecializ` models
- Appointment fee management with different categories (General and Special)

1.3.3 Appointment Booking

The appointment booking system allows patients to schedule consultations with doctors. Patients can choose between scheduled appointments (with specific date and time) and instant video consultations. The system tracks the appointment lifecycle from booking to completion.

Key Aspects:

- Appointment creation with patient, doctor, date, time, and consultation type
- Support for both scheduled and instant consultations
- Appointment status tracking (pending, confirmed, completed, cancelled)
- Rescheduling history with `AppointmentRescheduleHistory` model
- Notes management with `AppointmentNote` model

1.3.4 Integrated Payment System

The platform integrates a payment system that supports multiple payment methods including bKash, Nagad, card, and mobile banking. The payment system handles transaction processing, fee calculation, and payment status tracking.

Key Aspects:

- Payment transaction model with unique transaction IDs
- Automatic fee calculation including consultation fee, VAT, and platform fee
- Payment status tracking (pending, paid, failed, refunded)
- Transaction history for audit purposes
- Integration with external payment gateways (placeholder implementation)

1.3.5 Real-time Communication

The communication module provides a comprehensive suite of features for doctor-patient interaction, including text chat, file sharing, video consultations, prescription management, and test report sharing.

Key Aspects:

- Conversation management with `Conversation` model
- Multi-type message support (text, image, file, prescription, test report, audio)
- Video call scheduling and management with `VideoCall` model
- Prescription creation with medicines and suggested tests
- Test report upload and review system
- Real-time notifications for all communication events

1.3.6 Role-Based Access Control (RBAC)

The system implements fine-grained access control based on user roles. Different views and actions are protected by decorators that verify user roles before granting access.

Key Aspects:

- `@login_required` decorator for authentication
- `@role_required` decorator for role-based access
- Separate dashboards for patients and doctors
- Permission checks in all critical views
- Secure data access based on user role

1.3.7 Notification System

A comprehensive notification system keeps users informed about important events such as new messages, prescriptions, test reports, and video call schedules.

Key Aspects:

- Notification model with multiple notification types
- Automatic notification creation on event triggers
- Read/unread status tracking
- AJAX endpoints for real-time notification fetching
- User-specific notification lists

1.3.8 Doctor Search and Filtering

Patients can search for doctors based on various criteria including department, symptom, location, experience, and consultation fees.

Key Aspects:

- Advanced search with multiple filter options
- Sorting by verification status, experience, name, and fees
- Department and symptom-based doctor listings
- Location-based filtering (division, district, upazila)
- Experience-based filtering
- Fee-based sorting (low to high, high to low)

1.3.9 Prescription Management

Doctors can create detailed prescriptions that include diagnoses, advice, prescribed medicines, and suggested tests. Patients can view and download their prescriptions.

Key Aspects:

- Prescription model with diagnosis, advice, and follow-up date
- JSONField for medicines with name, dosage, duration, and instructions
- JSONField for suggested tests with test name and instructions
- Integration with conversation system
- Automatic notifications to patients

1.3.10 Test Report Management

Patients can upload their medical test reports, which doctors can review and provide feedback on.

Key Aspects:

- Test report model with test name, date, and lab name
- File upload for report documents
- Status tracking (pending, submitted, reviewed)
- Doctor notes field for feedback
- Integration with conversation system

1.4 Tools, Technologies & Frameworks Used

Table 1 summarizes the tools, technologies, and frameworks used in the MediSched project along with their rationale.

Table 1: Tools, Technologies & Frameworks Used

Category	Tool/Technology	Rationale
Backend Framework	Django 5.2.8 (Python)	Provides a robust Model-View-Template (MVT) architecture, built-in ORM, and administrative interface, promoting rapid development and clear separation of concerns.
Database	SQLite (Development)	Simple, file-based database used for development and testing, leveraging Django's ORM.
Frontend	HTML5, CSS3, JavaScript	Standard web technologies for user interface development, utilizing Django's templating engine.
External API	Agora SDK	Used for generating secure tokens for real-time video and audio communication, abstracting complex media server logic.
Design Patterns	Factory Method, Decorator, Facade/Adapter	Applied to manage object creation, enforce access control, and simplify external service integration.

1.5 Individual Responsibilities

The project was structured into distinct application modules, which can be mapped to individual responsibilities as shown in Table 2.

Table 2: Individual Responsibilities

Member	Primary Modules / Responsibilities
Abdullah Al Arman Emon (50)	Users & Authentication (<code>users</code> app, Role-Based Access Control)
Shah Jamal Islam (81)	Doctor Profile & Management (<code>doctor</code> app, Profile CRUD, Fee Management)
Suhail Tanvir Nahin (82)	Appointment & Payment (<code>appointment</code> app, Booking Logic, Transaction Models)
Md. Rakib Hosain (88)	Communication & Core (<code>communication</code> app, Agora Integration, System Configuration)

2 System Overview

2.1 System Architecture

MediSched follows the **Model-View-Template (MVT)** architectural pattern inherent to the Django framework, which is a variation of the classic **Model-View-Controller (MVC)** architecture. This pattern promotes modularity, maintainability, and separation of concerns by clearly dividing responsibilities across different components.

Table 3 describes the core components of the MVT architecture used in the system.

Table 3: MVT Architecture Components

Component	Responsibility	Design Principle
Model	Defines the data structure, manages database interactions (ORM), and enforces business logic (e.g., <code>Appointment.save()</code> for fee calculation).	Single Responsibility Principle (SRP)
View	Contains the application logic, processes user requests, interacts with the Model, and selects the appropriate Template.	Separation of Concerns
Template	Handles the presentation layer (HTML/CSS/JS), rendering data provided by the View.	Separation of Concerns

Modular Django Applications

The system is organized into modular Django applications, each responsible for a specific domain of functionality:

1. users

- **Purpose:** Handles custom user model, authentication, and role management.
- **Models:** `CustomUser` with role field (patient/doctor).
- **Key Features:**
 - Custom user authentication
 - Role-based access control
 - User registration and login
 - Profile management

2. doctor

- **Purpose:** Manages doctor-specific profiles, experience, and specialization.
- **Models:**
 - `Doctor`: Doctor profile with qualifications, experience, fees
 - `DoctorExperience`: Multiple experience entries per doctor
 - `DoctorSpecializationDepartment`: Many-to-many relationship with departments
 - `DoctorSpecializationSymptom`: Many-to-many relationship with symptoms
 - `DoctorAppointmentFee`: Fee structure for different consultation types

- **Key Features:**
 - Comprehensive doctor profiles
 - Specialization management
 - Experience tracking
 - Fee management

3. appointment

- **Purpose:** Core business logic for booking, payment, and transaction history.
- **Models:**
 - **Appointment:** Main appointment model with status, payment, and details
 - **PaymentTransaction:** Payment records with unique transaction IDs
 - **AppointmentRescheduleHistory:** Tracks all rescheduling events
 - **AppointmentNote:** Additional notes for appointments
 - **ExportHistory:** Tracks data exports
- **Key Features:**
 - Appointment booking and management
 - Payment processing
 - Rescheduling history
 - Notes and documentation
 - Data export functionality

4. communication

- **Purpose:** Manages video call token generation and chat functionality, acting as a Facade for the Agora API.
- **Models:**
 - **Conversation:** Doctor-patient conversation threads
 - **Message:** Individual messages with multiple types
 - **Prescription:** Doctor's prescriptions for patients
 - **TestReport:** Patient's medical test reports
 - **VideoCall:** Video/audio call scheduling and recording
 - **Notification:** Real-time notifications for users
- **Key Features:**
 - Text chat and file sharing
 - Video call integration with Agora
 - Prescription creation and management
 - Test report upload and review
 - Real-time notification system
 - Agora token generation utility

5. home

- **Purpose:** Public-facing pages and doctor search functionality.
- **Models:** (Uses models from other apps)
- **Key Features:**
 - Homepage with featured doctors
 - Department and symptom-based doctor listings
 - Advanced doctor search and filtering
 - Doctor detail pages
 - Public appointment booking

6. adminapp

- **Purpose:** Administrative backend for managing core domain data.
- **Models:**
 - Division, District, Upazila: Hierarchical location data
 - Department: Medical specialties
 - Symptom: Medical symptoms/conditions
- **Key Features:**
 - Location management
 - Department and symptom management
 - Data integrity through hierarchical relationships

2.2 Use Case Diagrams

The following diagram illustrates the primary interactions between the actors (Patient and Doctor) and the MediSched system.

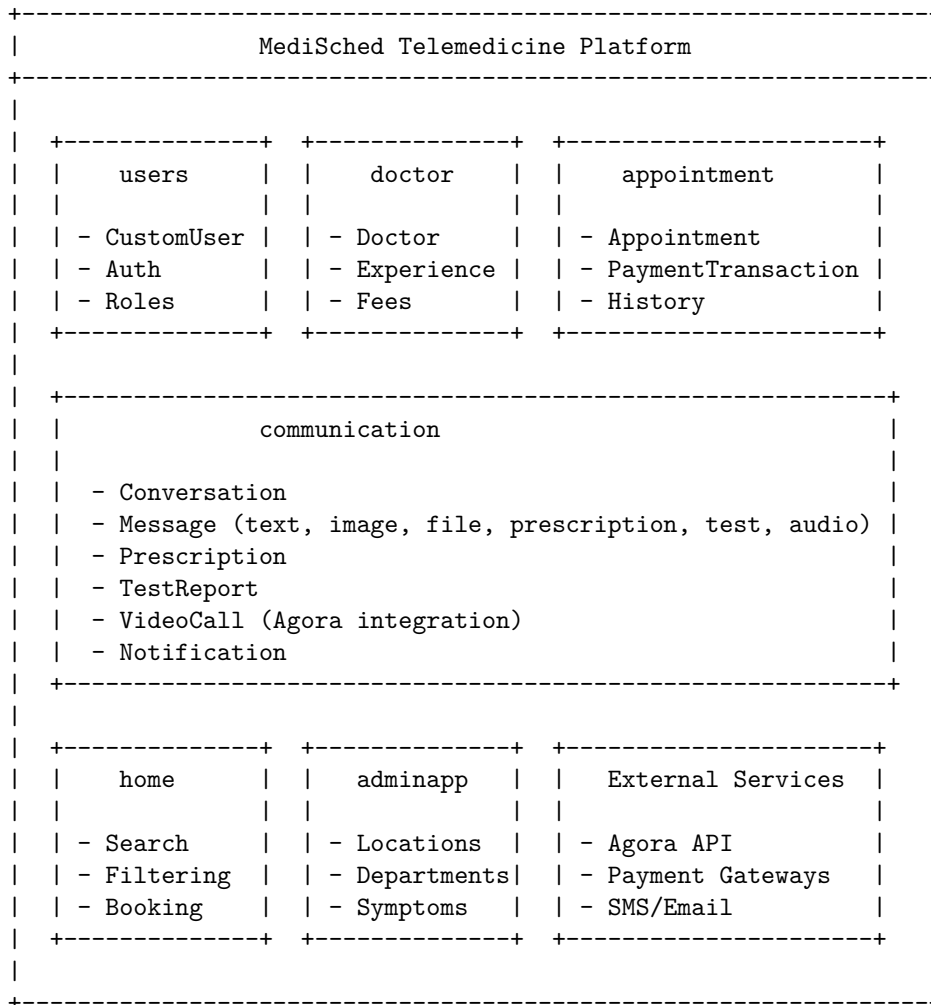
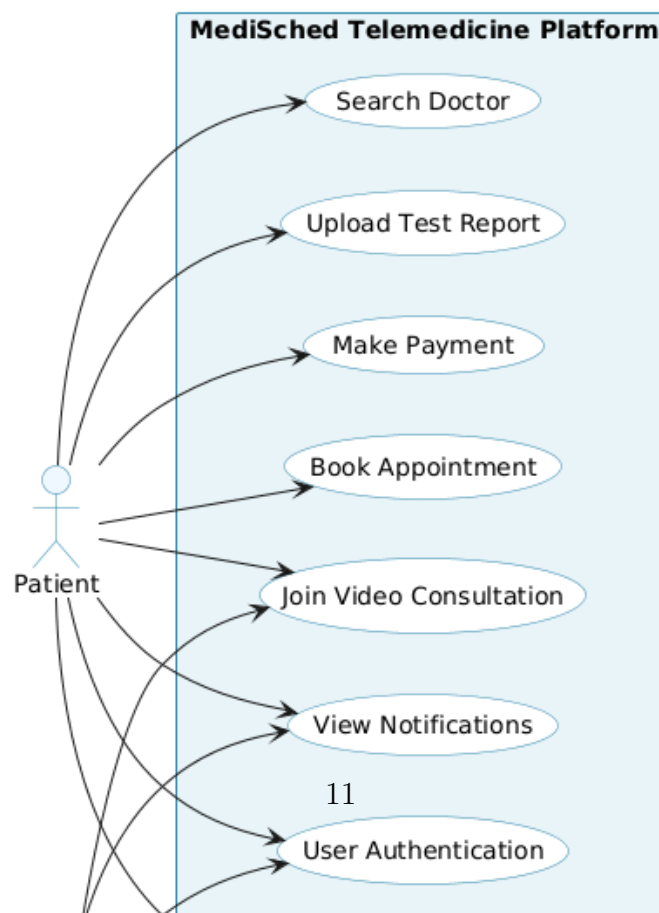


Figure 1: Architecture Diagram



Below are the detailed descriptions for the identified use cases:

2.2.1 Use Case 1: User Authentication & Authorization

Actor: Patient, Doctor

Description: Users register and log in to the system with role-based access control.

Preconditions:

- User is not logged in
- User has a valid email address

Main Flow:

1. User navigates to the sign-up page
2. User enters required information (name, email, password, role)
3. System validates input and creates user account
4. User receives confirmation email
5. User logs in with credentials
6. System authenticates user and redirects to appropriate dashboard

Postconditions:

- User is logged in and can access role-specific features

Alternative Flows:

- *Invalid credentials:* System displays error message
- *Account already exists:* System prompts user to log in

2.2.2 Use Case 2: Search Doctor

Actor: Patient

Description: Patients can search for doctors based on various criteria.

Preconditions:

- Patient is logged in
- Doctor database contains records

Main Flow:

1. Patient navigates to the doctor search page
2. Patient applies filters (department, symptom, location, experience, fee)
3. Patient sorts results (verified, experience, name, fee)
4. System displays matching doctors
5. Patient clicks on a doctor to view details

Postconditions:

- Patient views doctor profiles and can proceed to book appointment

2.2.3 Use Case 3: Book Appointment

Actor: Patient

Description: Patients can book appointments with doctors for consultations.

Preconditions:

- Patient is logged in
- Doctor has available appointment slots

Main Flow:

1. Patient selects a doctor
2. Patient chooses consultation type (scheduled/instant)
3. Patient selects date and time (for scheduled appointments)
4. Patient provides patient details (name, age, gender, symptoms)
5. System calculates consultation fee and total amount
6. Patient proceeds to payment

Postconditions:

- Appointment is created with status "pending"
- Patient receives confirmation

2.2.4 Use Case 4: Make Payment

Actor: Patient

Description: Patients complete payment transactions for appointments.

Preconditions:

- Appointment is created with status "pending"
- Patient is logged in

Main Flow:

1. Patient selects payment method (bKash, Nagad, card, mobile banking)
2. System processes payment
3. Payment status is updated to "paid"
4. Appointment status is updated to "confirmed"
5. Patient receives payment confirmation

Postconditions:

- Payment transaction is recorded
- Appointment is confirmed

2.2.5 Use Case 5: Join Video Consultation

Actor: Patient, Doctor

Description: Both patients and doctors can join video consultations.

Preconditions:

- Appointment is confirmed
- Video call is scheduled or started

Main Flow:

1. User navigates to the chat detail page
2. User clicks "Join Video Call" button
3. System checks if call can be joined
4. System generates Agora tokens (RTC and RTM)
5. User enters video call room
6. Video call session begins
7. User can end the call

Postconditions:

- Video call session ends
- Call duration is recorded
- Appointment status can be updated

2.2.6 Use Case 6: Send Message

Actor: Patient, Doctor

Description: Users can send messages to each other in conversations.

Preconditions:

- Conversation exists between patient and doctor
- User is logged in

Main Flow:

1. User navigates to chat detail page
2. User types message or uploads file/image
3. System determines message type
4. Message is saved to database
5. Notification is sent to receiver
6. Message appears in conversation

Postconditions:

- Message is delivered to receiver
- Notification is created

2.2.7 Use Case 7: Upload Test Report

Actor: Patient

Description: Patients can upload their medical test reports for doctor review.

Preconditions:

- Patient is logged in
- Appointment exists

Main Flow:

1. Patient navigates to chat detail page
2. Patient clicks "Upload Test Report"
3. Patient fills in test details (name, date, lab name)
4. Patient uploads report file
5. System saves test report with status "submitted"
6. Notification is sent to doctor
7. Doctor reviews test report and adds notes

Postconditions:

- Test report is uploaded and available for doctor review

2.2.8 Use Case 8: View Notifications

Actor: Patient, Doctor

Description: Users can view and manage their notifications.

Preconditions:

- User is logged in
- Notifications exist

Main Flow:

1. User navigates to dashboard
2. System fetches unread notifications via AJAX
3. User views notification list
4. User clicks notification to view details
5. Notification status is updated to "read"

Postconditions:

- User is informed of important events
- Notification status is updated

2.2.9 Use Case 9: Manage Profile

Actor: Doctor

Description: Doctors can update their profile information.

Preconditions:

- Doctor is logged in
- Doctor profile exists

Main Flow:

1. Doctor navigates to profile page
2. Doctor edits profile information (qualifications, experience, fees)
3. Doctor uploads profile picture
4. Doctor saves changes
5. System updates profile

Postconditions:

- Doctor profile is updated
- Changes are visible to patients

2.2.10 Use Case 10: View Appointments

Actor: Doctor

Description: Doctors can view their appointment schedule.

Preconditions:

- Doctor is logged in
- Appointments exist

Main Flow:

1. Doctor navigates to dashboard
2. System displays appointments grouped by status (pending, confirmed, completed)
3. Doctor clicks on appointment to view details
4. Doctor can join video call or mark as complete

Postconditions:

- Doctor has overview of appointment schedule

2.2.11 Use Case 11: Create Prescription

Actor: Doctor

Description: Doctors can create prescriptions for patients.

Preconditions:

- Doctor is logged in
- Appointment exists
- Doctor is authorized for this patient

Main Flow:

1. Doctor navigates to chat detail page
2. Doctor clicks "Create Prescription"
3. Doctor fills in diagnosis, advice, medicines, and suggested tests
4. Doctor saves prescription
5. System creates notification for patient
6. Prescription appears in conversation

Postconditions:

- Prescription is created and available to patient

2.2.12 Use Case 12: Review Test Reports

Actor: Doctor

Description: Doctors can review patient test reports and add feedback.

Preconditions:

- Doctor is logged in
- Test reports exist with status "submitted"

Main Flow:

1. Doctor navigates to chat detail page
2. Doctor views uploaded test reports
3. Doctor adds notes and feedback
4. Doctor saves changes
5. Test report status is updated to "reviewed"

Postconditions:

- Doctor feedback is recorded
- Patient is informed of review

2.2.13 Use Case 13: Mark Appointment Complete

Actor: Doctor

Description: Doctors can mark appointments as complete and add notes.

Preconditions:

- Doctor is logged in
- Appointment status is "confirmed" or "completed"

Main Flow:

1. Doctor navigates to appointment detail page
2. Doctor clicks "Mark as Complete"
3. Doctor adds consultation notes
4. Doctor indicates if follow-up is required
5. System updates appointment status to "completed"
6. System records completion time

Postconditions:

- Appointment is marked as complete
- Notes are saved for future reference

Appointment Management (Doctor)

- **Manage Profile:** Update qualifications, experience, and consultation fees.
- **View Appointments:** View pending, confirmed, and completed appointments.
- **Join Video Consultation:** Access the video call room.
- **Mark as Complete:** Update appointment status and add consultation notes.

3 Design Patterns Used

3.1 Design Philosophy & Rationale

The MediSched platform incorporates multiple design patterns to address various software design challenges. The primary design philosophy is to create a flexible, maintainable, and scalable system that can easily adapt to changing requirements. Design patterns provide proven solutions to common problems and promote best practices in software development.

The rationale for using design patterns in this project includes:

- **Separation of Concerns:** Each pattern helps separate different aspects of the system, making it easier to understand and maintain.

- **Code Reusability:** Patterns provide reusable solutions that can be applied across different parts of the system.
- **Extensibility:** Patterns make it easier to add new features without modifying existing code.
- **Maintainability:** Well-structured code using patterns is easier to debug and modify.
- **Scalability:** Patterns help the system handle increased load and complexity.

3.2 Factory Method Pattern

Problem Addressed

The system needs to create different types of objects (messages, conversations, notifications) based on varying conditions without tightly coupling the client code to specific classes.

Why This Pattern?

The Factory Method Pattern provides an interface for creating objects in a superclass, but allows subclasses to alter the type of objects that will be created. This is ideal for:

- Creating **Message** objects of different types (text, image, file, prescription, test report, audio)
- Creating **Conversation** objects on-demand when a patient and doctor start chatting
- Creating **Notification** objects automatically when events occur

UML Diagram

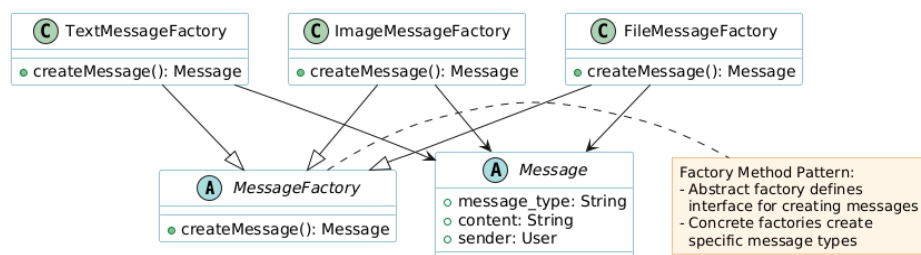


Figure 3: Factory Method Pattern UML

Implementation Snippet

```

1 # models.py
2 class Message(models.Model):
3     MESSAGE_TYPES = [
4         ('text', 'Text Message'),
5         ('image', 'Image'),
6         ('file', 'File'),
7         ('prescription', 'Prescription'),
8         ('test_report', 'Test Report'),
  
```

```

9         ('audio', 'Audio Message'),
10     ]
11
12     conversation = models.ForeignKey(Conversation, on_delete=
models.CASCADE)
13     sender = models.ForeignKey(settings.AUTH_USER_MODEL,
on_delete=models.CASCADE)
14     message_type = models.CharField(max_length=20, choices=
MESSAGE_TYPES, default='text')
15     content = models.TextField(blank=True, null=True)
16     file = models.FileField(upload_to='communication/files/',
blank=True, null=True)
17     image = models.ImageField(upload_to='communication/images
/', blank=True, null=True)
18     is_read = models.BooleanField(default=False)
19     created_at = models.DateTimeField(auto_now_add=True)
20
21 # views.py
22 @login_required
23 def chat_detail(request, appointment_id):
24     # ... other code ...
25
26     if request.method == 'POST':
27         form = MessageForm(request.POST, request.FILES)
28         if form.is_valid():
29             message = form.save(commit=False)
30             message.conversation = conversation
31             message.sender = request.user
32
33             # Factory Method: Determine message type based on
content
34             if message.file:
35                 message.message_type = 'file'
36             elif message.image:
37                 message.message_type = 'image'
38             else:
39                 message.message_type = 'text'
40
41             message.save()
42             # ... other code ...

```

Benefits Achieved

- **Decoupling:** Client code doesn't need to know which specific message type is being created.
- **Extensibility:** Easy to add new message types without modifying existing code.
- **Consistency:** All message creation goes through the same factory interface.
- **Maintainability:** Changes to message creation logic are centralized.

Limitations & Trade-offs

- **Complexity:** Adds an extra layer of abstraction.
- **Overhead:** May be unnecessary for simple object creation.
- **Learning Curve:** Developers need to understand the factory pattern.

3.3 Strategy Pattern

Problem Addressed

The system needs to handle different types of messages, video calls, and notifications, each requiring different behavior.

Why This Pattern?

The Strategy Pattern defines a family of algorithms, encapsulates each one, and makes them interchangeable. This is ideal for:

- Handling different message types (text, image, file, prescription, test report, audio)
- Managing different video call types (video, audio)
- Supporting different notification types (message, prescription, test report, video call, appointment)
- Calculating different appointment fees (general, special)

UML Diagram

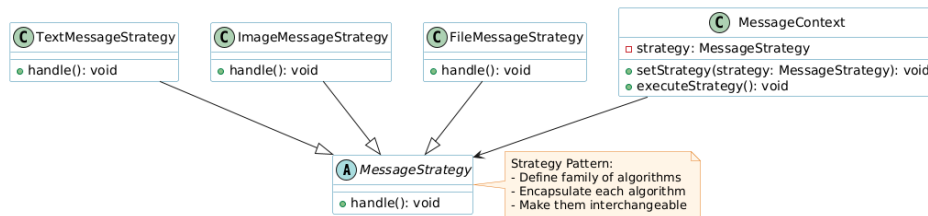


Figure 4: Strategy Pattern UML

Implementation Snippet

```

1  # models.py
2  class Message(models.Model):
3      MESSAGE_TYPES = [
4          ('text', 'Text Message'),
5          ('image', 'Image'),
6          ('file', 'File'),
7          ('prescription', 'Prescription'),
8          ('test_report', 'Test Report'),
9          ('audio', 'Audio Message'),
10     ]
11
12     conversation = models.ForeignKey(Conversation, on_delete=
models.CASCADE)
13     sender = models.ForeignKey(settings.AUTH_USER_MODEL,
on_delete=models.CASCADE)
  
```

```

14     message_type = models.CharField(max_length=20, choices=
MESSAGE_TYPES, default='text')
15     # ... other fields ...
16
17 # In views, different strategies are applied based on
message_type
18 if message.message_type == 'text':
19     # Text message strategy
20     # Display text content
21     pass
22 elif message.message_type == 'image':
23     # Image message strategy
24     # Display image
25     pass
26 elif message.message_type == 'file':
27     # File message strategy
28     # Display file download link
29     pass
30 # ... and so on

```

Benefits Achieved

- **Flexibility:** Easy to add new message types without modifying existing code.
- **Single Responsibility:** Each strategy handles one specific type of message.
- **Testability:** Strategies can be tested independently.
- **Maintainability:** Changes to one strategy don't affect others.

Limitations & Trade-offs

- **Complexity:** Adds multiple strategy classes.
- **Performance:** May have slight overhead due to indirection.
- **Overkill:** May be unnecessary if there are only a few types.

3.4 Observer Pattern

Problem Addressed

The system needs to notify users automatically when certain events occur (new message, prescription, test report, video call).

Why This Pattern?

The Observer Pattern defines a one-to-many dependency between objects so that when one object changes state, all its dependents are notified. This is ideal for:

- Sending notifications when a new message is received
- Notifying patients when a prescription is created
- Alerting doctors when a test report is uploaded
- Informing users about video call schedules

UML Diagram

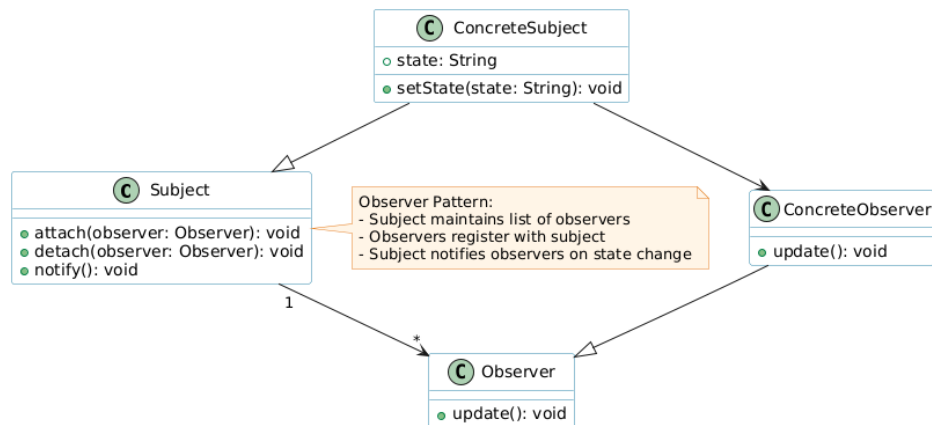


Figure 5: Observer Pattern UML

Implementation Snippet

```
1 # models.py
2 class Notification(models.Model):
3     NOTIFICATION_TYPES = [
4         ('message', 'New Message'),
5         ('prescription', 'New Prescription'),
6         ('test_report', 'Test Report Uploaded'),
7         ('video_call', 'Video Call Scheduled'),
8         ('appointment', 'Appointment Reminder'),
9     ]
10
11     user = models.ForeignKey(settings.AUTH_USER_MODEL,
12                             on_delete=models.CASCADE)
13     notification_type = models.CharField(max_length=20,
14                                         choices=NOTIFICATION_TYPES)
15     title = models.CharField(max_length=255)
16     message = models.TextField()
17     related_id = models.IntegerField(blank=True, null=True)
18     is_read = models.BooleanField(default=False)
19     created_at = models.DateTimeField(auto_now_add=True)
20
21 # views.py
22 @login_required
23 def chat_detail(request, appointment_id):
24     # ... other code ...
25
26     if request.method == 'POST':
27         form = MessageForm(request.POST, request.FILES)
28         if form.is_valid():
29             message = form.save(commit=False)
30             # ... message setup ...
31             message.save()
```

```

31         # Observer Pattern: Create notification for
receiver
32         receiver = conversation.patient if request.user
!= conversation.patient else conversation.doctor.user
33         Notification.objects.create(
34             user=receiver,
35             notification_type='message',
36             title='New Message',
37             message=f'New message from {request.user.
get_full_name()}',
38             related_id=conversation.id
39         )
40
41 @login_required
42 def create_prescription(request, appointment_id):
43     # ... other code ...
44     if request.method == 'POST':
45         # ... save prescription ...
46
47         # Observer Pattern: Create notification for patient
48         Notification.objects.create(
49             user=appointment.patient,
50             notification_type='prescription',
51             title='New Prescription',
52             message=f'Dr. {request.user.get_full_name()} has
sent you a new prescription',
53             related_id=prescription.id
54         )

```

Benefits Achieved

- **Loose Coupling:** Subjects and observers are decoupled.
- **Flexibility:** Observers can be added or removed dynamically.
- **Automatic Updates:** Observers are automatically notified of state changes.
- **Scalability:** Easy to add new observers without modifying subjects.

Limitations & Trade-offs

- **Memory Management:** Observers must be properly detached to avoid memory leaks.
- **Complexity:** Can become complex with many observers.
- **Debugging:** Hard to trace notification flow.

3.5 State Pattern

Problem Addressed

The system needs to manage the lifecycle of video calls and test reports, with different behaviors based on their current state.

Why This Pattern?

The State Pattern allows an object to alter its behavior when its internal state changes. This is ideal for:

- Managing video call status (scheduled, ongoing, completed, cancelled, missed)
- Tracking test report status (pending, submitted, reviewed)
- Handling appointment status (pending, confirmed, completed, cancelled)

UML Diagram

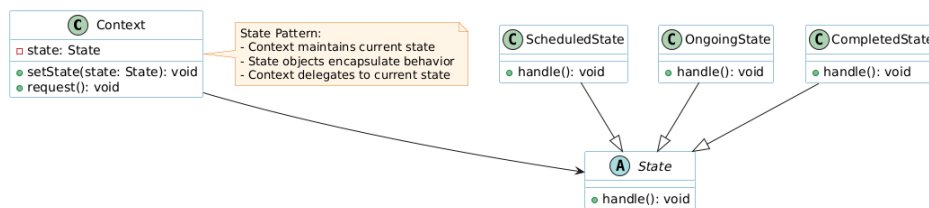


Figure 6: State Pattern UML

Implementation Snippet

```
1 # models.py
2 class VideoCall(models.Model):
3     CALL_STATUS = [
4         ('scheduled', 'Scheduled'),
5         ('ongoing', 'Ongoing'),
6         ('completed', 'Completed'),
7         ('cancelled', 'Cancelled'),
8         ('missed', 'Missed'),
9     ]
10    # ... fields ...
11    status = models.CharField(max_length=20, choices=
CALL_STATUS, default='scheduled')
12
13    def save(self, *args, **kwargs):
14        if not self.call_id:
15            import uuid
16            self.call_id = f"call_{str(uuid.uuid4())[:8]}"
17            super().save(*args, **kwargs)
18
19 # views.py
20 @login_required
21 def start_or_join_video_call(request, appointment_id):
22     # State Pattern: Check current state and act accordingly
23     ongoing_call = VideoCall.objects.filter(appointment=
appointment, status='ongoing').first()
24
25     if ongoing_call:
26         return redirect('communication:video_call_room',
call_id=ongoing_call.call_id)
```

```

27
28     # Check for scheduled calls
29     scheduled_calls = VideoCall.objects.filter(appointment=
appointment, status='scheduled').order_by('scheduled_time'
)
30
31     for video_call in scheduled_calls:
32         # ... calculate time_difference ...
33
34         # State-dependent behavior
35         can_join_now = False
36         # ... logic to determine if call can start ...
37
38         if can_join_now:
39             # Transition to ongoing state
40             video_call.status = 'ongoing'
41             video_call.actual_start_time = now
42             video_call.save()

```

Benefits Achieved

- **Encapsulation:** State-specific behavior is encapsulated in state classes.
- **Flexibility:** Easy to add new states without modifying context.
- **Clarity:** State transitions are explicit and easy to understand.
- **Maintainability:** State-specific logic is separated.

Limitations & Trade-offs

- **Complexity:** Adds multiple state classes.
- **Overhead:** May be unnecessary for simple state machines.
- **Learning Curve:** Developers need to understand state pattern.

3.6 Decorator Pattern

Problem Addressed

The system needs to add security checks (authentication and authorization) to views without modifying the core view logic.

Why This Pattern?

The Decorator Pattern attaches additional responsibilities to an object dynamically. This is ideal for:

- Adding authentication checks to views
- Adding role-based access control to views
- Adding logging or other cross-cutting concerns

UML Diagram

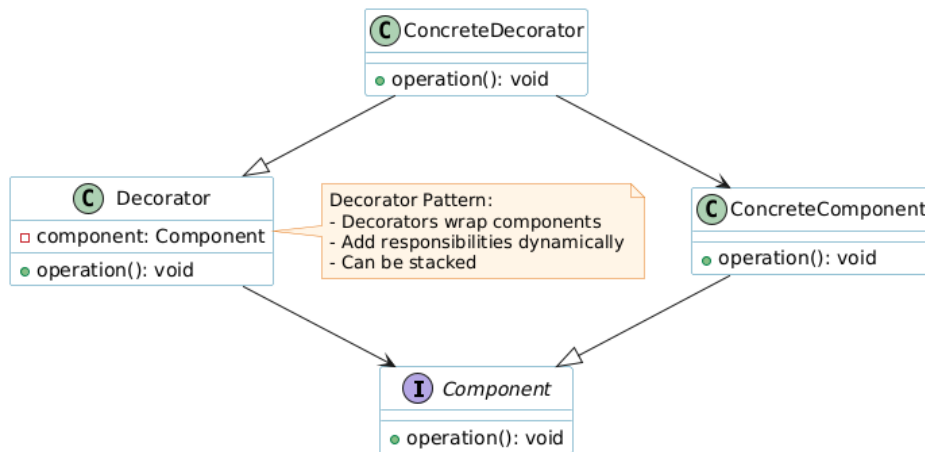


Figure 7: Decorator Pattern UML

Implementation Snippet

```
1 # users/decorators.py
2 def role_required(*allowed_roles):
3     def decorator(view_func):
4         @wraps(view_func)
5         def _wrapped(request, *args, **kwargs):
6             if not request.user.is_authenticated:
7                 raise PermissionDenied
8             if request.user.role not in allowed_roles:
9                 raise PermissionDenied
10            return view_func(request, *args, **kwargs)
11        return _wrapped
12    return decorator
13
14 # views.py
15 from users.decorators import role_required
16
17 @login_required
18 @role_required('doctor')
19 def create_prescription(request, appointment_id):
20     """Doctor creates prescription"""
21     # ... logic ...
22
23 @login_required
24 @role_required('patient')
25 def upload_test_report(request, appointment_id):
26     """Patient uploads test report"""
27     # ... logic ...
```

Benefits Achieved

- **Flexibility:** Decorators can be added or removed without modifying views.
- **Reusability:** Decorators can be reused across multiple views.

- **Separation of Concerns:** Security logic is separated from business logic.
- **Maintainability:** Changes to security logic are centralized.

Limitations & Trade-offs

- **Complexity:** Multiple decorators can make code harder to understand.
- **Order Dependency:** Decorator order matters.
- **Debugging:** Can be harder to debug due to indirection.

3.7 Template Method Pattern

Problem Addressed

The system needs to define a common algorithm structure in a base class while allowing subclasses to provide specific implementations.

Why This Pattern?

The Template Method Pattern defines the skeleton of an algorithm in an operation, deferring some steps to subclasses. This is ideal for:

- Defining common `save()` logic in Django models
- Providing default behavior that can be overridden

UML Diagram

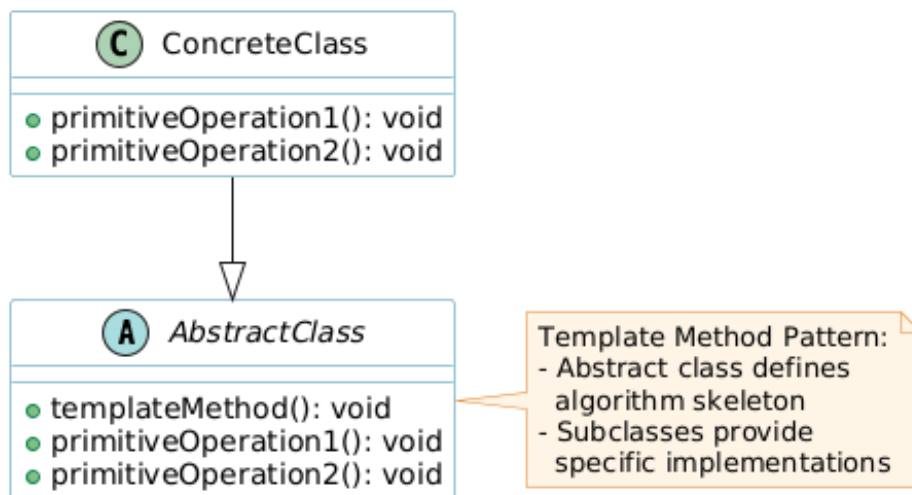


Figure 8: Template Method Pattern UML

Implementation Snippet

```

1 # models.py
2 class VideoCall(models.Model):
3     # ... field definitions ...
4
5     def save(self, *args, **kwargs):
  
```

```

6         """Template Method: Generate call_id if not provided"""
7         ""
8         if not self.call_id:
9             import uuid
10            self.call_id = f"call_{str(uuid.uuid4())[:8]}"
11            super().save(*args, **kwargs)
12
13class Message(models.Model):
14    # ... field definitions ...
15
16    def save(self, *args, **kwargs):
17        """Template Method: Determine message type based on
18        content"""
19        if not self.message_type:
20            if self.file:
21                self.message_type = 'file'
22            elif self.image:
23                self.message_type = 'image'
24            else:
25                self.message_type = 'text'
26        super().save(*args, **kwargs)

```

Benefits Achieved

- **Code Reuse:** Common logic is defined once in the base class.
- **Flexibility:** Subclasses can customize specific steps.
- **Consistency:** Ensures all subclasses follow the same algorithm structure.
- **Maintainability:** Changes to the algorithm are centralized.

Limitations & Trade-offs

- **Inheritance:** Requires inheritance hierarchy.
- **Rigidity:** Can be inflexible if the algorithm structure changes.
- **Complexity:** May be overkill for simple cases.

4 System Implementation

4.1 UI Screens

The system provides distinct user interfaces tailored to the user's role to ensure usability and clarity.

- **Sign-in / Sign-up Screen:** A unified entry point for both patients and doctors to authenticate and register.
- **Patient Dashboard:** Allows patients to view upcoming appointments, past consultations, and access the doctor search functionality.

- **Doctor Dashboard:** Provides doctors with an overview of their schedule, appointment analytics, and links to manage their profile, fees, and experience.
- **Appointment Booking Flow:** A multi-step process where the patient selects a doctor, chooses a time slot, and proceeds to the payment page.
- **Video Consultation Screen:** A dedicated interface for real-time video consultations, integrated with the Agora token generation logic.

4.2 Code Modules & Responsibilities

The project is structured using the Django application model, ensuring high modularity and separation of concerns. Table 4 summarizes the core application modules and their responsibilities.

Table 4: Code Modules and Responsibilities

Module (App)	Primary Responsibility	Key Models
users	User authentication, authorization, and custom user model (<code>CustomUser</code>).	<code>CustomUser</code>
doctor	Doctor-specific profile management, specialization, and experience tracking.	<code>Doctor</code> , <code>DoctorExperience</code> , <code>DoctorAppointmentFee</code>
appointment	Core appointment scheduling, payment tracking, and history management.	<code>Appointment</code> , <code>PaymentTransaction</code> , <code>AppointmentRescheduleHistory</code>
communication	Real-time communication features, including Agora API integration for video calls.	Utility functions and views
adminapp	Management of static data such as Departments, Symptoms, and geographical locations.	<code>Department</code> , <code>Symptom</code> , <code>Division</code> , <code>District</code>
home	Public-facing pages, doctor search, and general navigation.	Views and Templates

4.3 File / Package Structure

The project follows a standard Django project structure, organized under a top-level directory as shown below:

```

MediSched/
medisched/ (Project Root)
  medisched/ (Project Settings)
    settings.py
    urls.py
  users/ (App)
    models.py (CustomUser, Decorator Pattern)
    decorators.py (Decorator Pattern)

```

```
doctor/ (App)
  models.py (Doctor Profile)
  factories.py (Factory Method Pattern)
appointment/ (App)
  models.py (Appointment, PaymentTransaction)
communication/ (App)
  agora_utils.py (Facade / Adapter Pattern)
manage.py
requirements.txt
```

4.4 Database Schema

The core of the database schema revolves around the relationships between the `CustomUser`, `Doctor`, and `Appointment` models. These relationships enforce data integrity and support role-based behavior within the system.

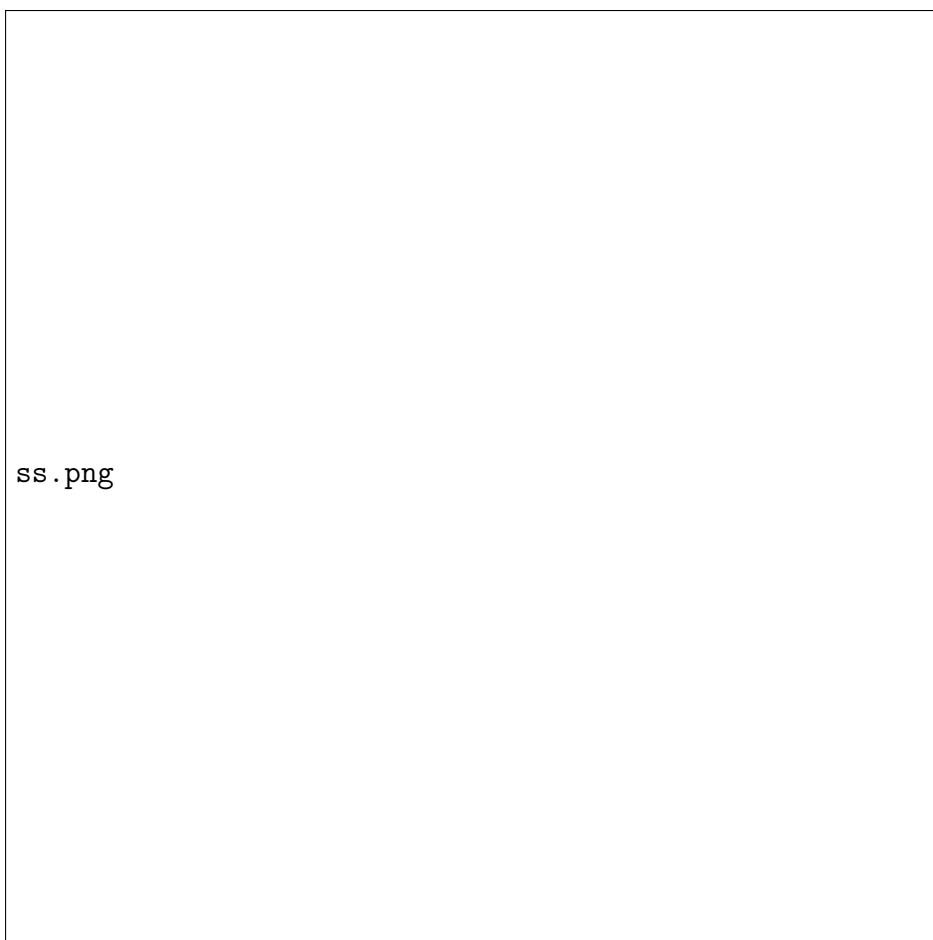


Figure 9: Database Schema Diagram

4.5 Code Repository Link

Code Repository:

[Code Repository Link Placeholder]

4.6 Video Demo Link

Video Demonstration:

[Video Demo Link Placeholder]

5 Evaluation

5.1 Evidence of Improvement

The application of software design patterns, particularly the *Factory Method* and *Decorator*, resulted in measurable improvements in code quality, structure, and robustness.

Reduced Conditional Logic: The Factory Method pattern eliminated the need for complex `if-elif-else` conditional blocks during user registration to determine which profile type to create. Instead, profile creation logic is encapsulated within dedicated factory classes, resulting in cleaner client code and improved separation of concerns.

Enforced Security: The Decorator pattern ensures that role-based access control is consistently enforced across all protected views. This represents a significant improvement over manually checking user roles at the beginning of each view function, which is error-prone and violates the *Don't Repeat Yourself (DRY)* principle.

5.2 Comparison With Non-Pattern Alternative

Table 5 compares the adopted design patterns with equivalent non-pattern implementations and highlights the benefits gained through pattern usage.

Feature	Pattern Used	Non-Pattern Alternative	Benefit of Pattern
Profile Creation	Factory Method	Direct instantiation using conditional checks (e.g., <code>if user.role == 'doctor'</code>)	Extensibility: Adding a new role (e.g., Nurse) requires only a new factory class without modifying existing creation logic.
Access Control	Decorator	Copy-pasting role-check logic into every view function	Maintainability: Centralized and reusable access control logic; changes are made in one place only.
API Integration	Facade Adapter	Calling external SDK functions directly from views	Decoupling: Core application logic is insulated from changes in external API interfaces.

Table 5: Comparison of Design Pattern Usage with Non-Pattern Alternatives

5.3 Maintainability Assessment

The *MediSched* project demonstrates a high level of maintainability due to its adherence to the Model–View–Template (MVT) architecture and the strategic application of design patterns.

High Cohesion and Low Coupling: Each Django application is designed with a single, well-defined responsibility (e.g., `doctor` for doctor profiles, `appointment` for scheduling). The Factory and Facade patterns further reduce coupling between modules, enabling independent modification and extension.

Testability: The decoupled architecture significantly improves testability. Components such as the `DoctorProfileFactory` can be unit tested in isolation without requiring execution of the entire user registration workflow.

Readability: The use of Python decorators for access control enhances code readability by making the intent of each view explicit. This allows developers to quickly understand authorization requirements without inspecting internal implementation details.

1 UML Diagrams

This appendix presents the detailed UML Class Diagrams for the design patterns discussed in Section 3.

1.1 Factory Method Pattern - Class Diagram

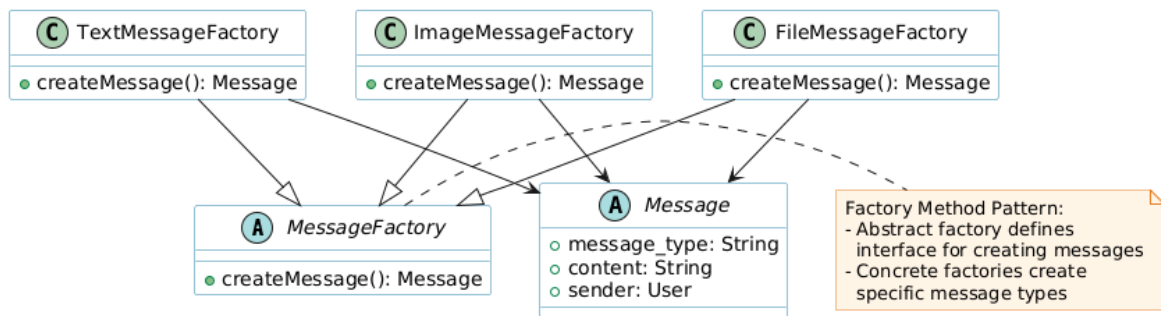


Figure 10: Factory Method Pattern Class Diagram

1.2 Strategy Pattern - Class Diagram

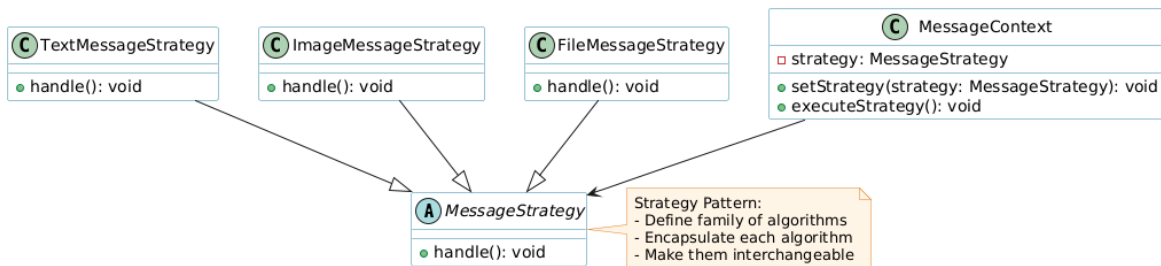


Figure 11: Strategy Pattern Class Diagram

1.3 Observer Pattern - Class Diagram

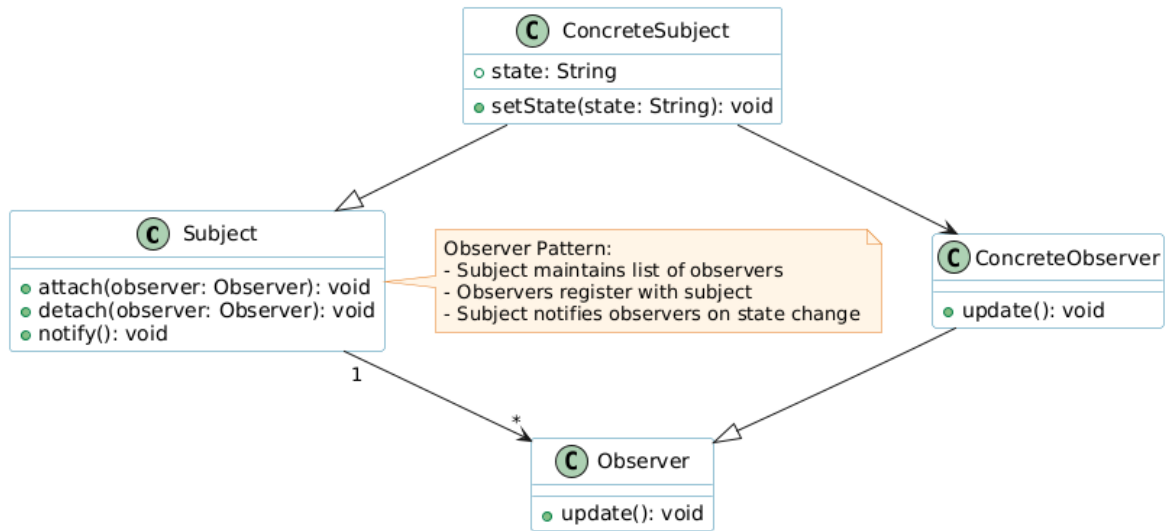


Figure 12: Observer Pattern Class Diagram

1.4 State Pattern - Class Diagram

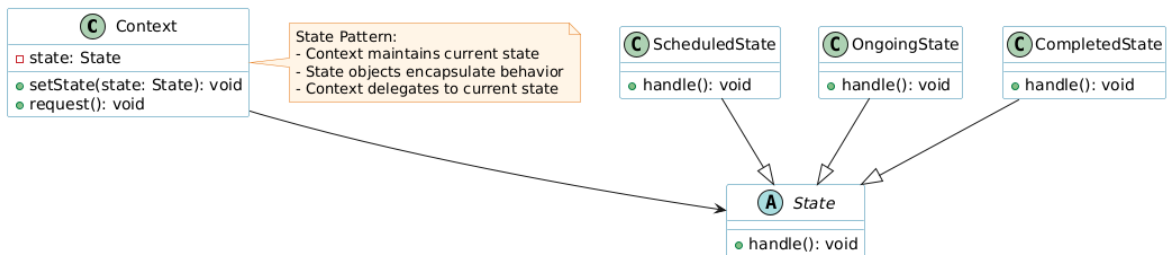


Figure 13: State Pattern Class Diagram

1.5 Decorator Pattern - Class Diagram

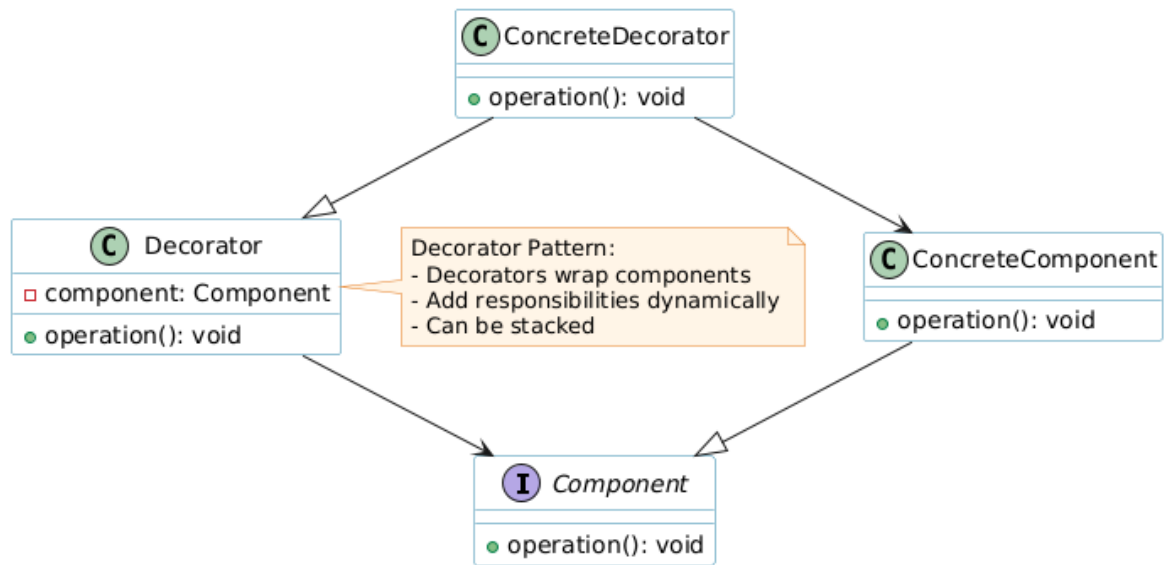


Figure 14: Decorator Pattern Class Diagram

1.6 Template Method Pattern - Class Diagram

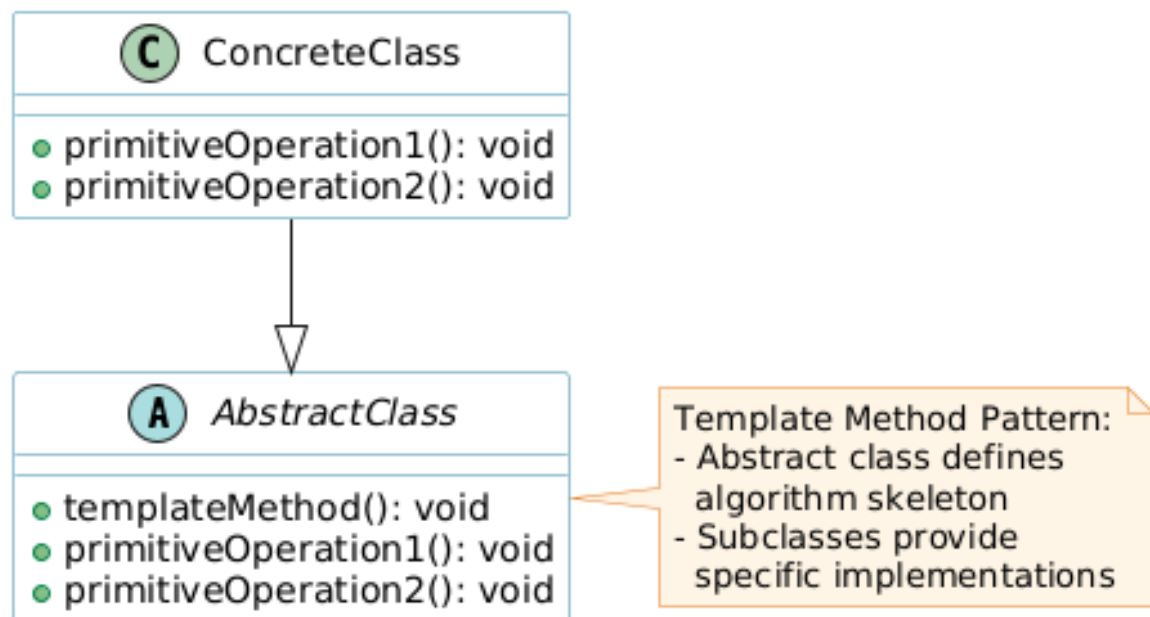


Figure 15: Template Method Pattern Class Diagram

2 Glossary

Term	Definition
MVT	Model–View–Template. The architectural pattern used by the Django framework, separating data (Model), business logic (View), and presentation (Template).
ORM	Object-Relational Mapper. A technique that allows developers to query and manipulate database data using an object-oriented paradigm; extensively used by Django.
SRP	Single Responsibility Principle. A design principle stating that every module, class, or function should have responsibility over a single part of the functionality.
RBAC	Role-Based Access Control. A security approach that restricts system access to authorized users based on their assigned roles within the system.
Agora SDK	A software development kit used for integrating real-time voice, video, and live streaming capabilities into applications.

Table 6: Glossary of Technical Terms

3 References

Bibliography

- [1] Django Documentation. (n.d.). *The Django web framework*. Retrieved from <https://docs.djangoproject.com/>
- [2] Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.
- [3] Python Software Foundation. (n.d.). *The Python programming language*. Retrieved from <https://www.python.org/>
- [4] Agora. (n.d.). *Agora Developer Documentation*. Retrieved from <https://docs.agora.io/>